Here's a **complete explanation** of **Instantiation** in Python OOP:

🎯 **WHAT IS INSTANTIATION?**

Instantiation = Process of creating an

OBJECT from a CLASS

Class → Blueprint / Template

Object → Real thing created from blueprint

Simple analogy:

Class → 🏠 House Blueprint (plan on paper)

Object → 🏡 Actual House built from that plan

You can build MULTIPLE houses

from ONE blueprint — same way

you can create MULTIPLE objects

from ONE class

🔑 **KEY TERMS**

Class → Template/Blueprint

Object → Instance of a class

Instance → Another word for Object

Instantiation → Act of creating an object

These mean the SAME thing:

obj = MyClass()

→ "obj is an OBJECT of MyClass"

→ "obj is an INSTANCE of MyClass"

→ "We INSTANTIATED MyClass"

→ "We created an INSTANCE of MyClass"

🏗️ HOW INSTANTIATION WORKS INTERNALLY

When you write:

obj = MyClass()

Python does these steps:

STEP 1 → Calls __new__()

Creates empty object in memory

Allocates memory space

STEP 2 → Calls __init__()

Fills the object with data

Sets up initial state

STEP 3 → Returns the object

Assigns it to variable 'obj'

Visual:

```
MyClass()
   |
   ▼
┌──────────────┐
| __new__() | → Creates empty object
|           |   in memory
└──────┬───────┘
      |
      ▼
┌──────────────┐
| __init__() | → Fills object with
|           |   attributes & values
└──────┬───────┘
      |
```

▼

obj = 🟢 Ready to use Object!

📝 **BASIC INSTANTIATION EXAMPLE**

python

```python
# Step 1 → Define a CLASS (Blueprint)
class Dog:

    def __init__(self, name, breed, age):
        # These are INSTANCE VARIABLES
        # Each object gets its OWN copy
        self.name  = name
        self.breed = breed
        self.age   = age

    def bark(self):
        return f"{self.name} says: Woof!"

    def info(self):
        return f"{self.name} is a {self.breed}, age {self.age}"


# Step 2 → INSTANTIATE (Create Objects)
dog1 = Dog("Tommy", "Labrador", 3)   # object 1
dog2 = Dog("Bruno", "Poodle",   5)   # object 2
dog3 = Dog("Max",   "Husky",    2)   # object 3

# Step 3 → USE the objects
print(dog1.name)       # Tommy
print(dog2.breed)      # Poodle
print(dog3.age)        # 2
```

```
print(dog1.bark())     # Tommy says: Woof!
print(dog2.info())     # Bruno is a Poodle, age 5
```

🔬 **DEEP DIVE — __new__ vs __init__**

python

```
class MyClass:

    # __new__ → CREATES the object
    # Called FIRST
    # Returns the new object
    def __new__(cls):
        print("__new__ called → Creating object")
        instance = super().__new__(cls)
        return instance

    # __init__ → INITIALIZES the object
    # Called SECOND
    # Fills object with data
    def __init__(self):
        print("__init__ called → Setting up object")
        self.value = 100


# When you instantiate:
obj = MyClass()

# Output:
# __new__ called → Creating object
# __init__ called → Setting up object
```

```
print(obj.value)  # 100
```

🧬 **INSTANCE VARIABLES vs CLASS VARIABLES**

python

```
class Student:

    # CLASS variable → Shared by ALL objects
    school = "ABC School"
    count  = 0

    def __init__(self, name, marks):
        # INSTANCE variables → Unique to each object
        self.name  = name
        self.marks = marks
        Student.count += 1   # increment shared count


# Creating instances
s1 = Student("Rahul", 90)
s2 = Student("Priya", 85)
s3 = Student("Amit",  78)

# Instance variables → different for each
print(s1.name)    # Rahul
print(s2.name)    # Priya
print(s3.name)    # Amit

# Class variable → SAME for all
print(s1.school)  # ABC School
print(s2.school)  # ABC School
print(s3.school)  # ABC School
```

```
# Count shared across all instances
print(Student.count)  # 3
```

🔄 MULTIPLE INSTANTIATION

python

```
class BankAccount:

    bank = "SBI"   # class variable

    def __init__(self, holder, balance):
        self.holder  = holder    # instance variable
        self.balance = balance   # instance variable
        self._pin    = 0000      # protected
        self.__id    = id(self)  # private (unique ID)

    def deposit(self, amount):
        self.balance += amount
        return f"Deposited ₹{amount}. New balance: ₹{self.balance}"

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient funds!"
        self.balance -= amount
        return f"Withdrawn ₹{amount}. New balance: ₹{self.balance}"

    def __str__(self):
        return f"Account[{self.holder}] → ₹{self.balance}"


# Creating MULTIPLE instances from ONE class
```

```
acc1 = BankAccount("Rahul", 50000)
acc2 = BankAccount("Priya", 75000)
acc3 = BankAccount("Amit",  30000)

# Each object is INDEPENDENT
print(acc1)             # Account[Rahul] → ₹50000
print(acc2)             # Account[Priya] → ₹75000
print(acc3)             # Account[Amit]  → ₹30000

# Actions on one DON'T affect others
print(acc1.deposit(10000))    # ₹60000
print(acc2.withdraw(5000))    # ₹70000
print(acc3)             # Still ₹30000 — unchanged

# All share class variable
print(acc1.bank)          # SBI
print(acc2.bank)          # SBI
```

🧪 **INSTANTIATION IN SELENIUM (Your Domain!)**

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Instantiating WebDriver
driver = webdriver.Chrome()   # ChromeDriver object

# Instantiating WebDriverWait
wait = WebDriverWait(driver, 10)  # Wait object
```

```
# Instantiating Page class (POM)
class LoginPage:
    def __init__(self, driver):
        self.driver   = driver
        self.username = (By.ID, "username")
        self.password = (By.ID, "password")
        self.btn      = (By.ID, "loginBtn")

    def login(self, user, pwd):
        self.driver.find_element(*self.username).send_keys(user)
        self.driver.find_element(*self.password).send_keys(pwd)
        self.driver.find_element(*self.btn).click()



# INSTANTIATION in test
driver   = webdriver.Chrome()         # driver instance
driver.get("https://example.com")

login_page = LoginPage(driver)        # page instance
login_page.login("admin", "pass123")  # use the instance
```

🏭 **DIFFERENT WAYS TO INSTANTIATE**

Way 1 — Normal Instantiation:

python

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age  = age

p = Person("Rahul", 25)   # standard way
```

Way 2 — Class Method (Alternative Constructor):

python

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age  = age

    @classmethod
    def from_string(cls, data):
        # data = "Rahul-25"
        name, age = data.split("-")
        return cls(name, int(age))   # creates instance

    @classmethod
    def from_dict(cls, data):
        # data = {"name": "Priya", "age": 22}
        return cls(data["name"], data["age"])


# Different ways to create same object
p1 = Person("Rahul", 25)            # normal
p2 = Person.from_string("Priya-22")  # from string
p3 = Person.from_dict({"name": "Amit", "age": 30})

print(p1.name)   # Rahul
print(p2.name)   # Priya
print(p3.name)   # Amit
```

Way 3 — List of Instances:

python

```
class Product:
    def __init__(self, name, price):
        self.name  = name
```

```
        self.price = price

# Creating multiple instances in a list
products = [
    Product("Laptop",  55000),
    Product("Mobile",  15000),
    Product("Tablet",  25000),
    Product("Monitor", 12000),
]

# Access all instances
for p in products:
    print(f"{p.name} → ₹{p.price}")

# Output:
# Laptop  → ₹55000
# Mobile  → ₹15000
# Tablet  → ₹25000
# Monitor → ₹12000
```

Way 4 — Dynamic Instantiation:

python

```
class Circle:
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle:
    def __init__(self, width, height):
        self.width  = width
        self.height = height
```

```
    def area(self):
        return self.width * self.height

# Dynamically choose which class to instantiate
def create_shape(shape_type, *args):
    shapes = {
        "circle"    : Circle,
        "rectangle" : Rectangle
    }
    return shapes[shape_type](*args)  # dynamic instantiation

c = create_shape("circle", 5)
r = create_shape("rectangle", 4, 6)

print(c.area())   # 78.5
print(r.area())   # 24
```

🔍 **CHECKING INSTANCES**

python

```
class Animal:
    pass

class Dog(Animal):
    pass

class Cat(Animal):
    pass

d = Dog()
c = Cat()
```

```
# isinstance() → checks if object is instance of class
print(isinstance(d, Dog))     # True
print(isinstance(d, Animal))  # True  (parent class)
print(isinstance(d, Cat))     # False

# type() → returns exact class
print(type(d))              # <class '__main__.Dog'>
print(type(d) == Dog)        # True
print(type(d) == Animal)      # False (exact match only)

# id() → unique memory address of each instance
d1 = Dog()
d2 = Dog()
print(id(d1))   # some memory address
print(id(d2))   # DIFFERENT memory address
print(d1 is d2) # False → different objects
```

📊 **SUMMARY TABLE**

Term	Meaning	Example
Class	Blueprint	class Dog:
Instantiation	Creating object	Dog()
Instance/Object	Created object	dog1 = Dog()
__new__	Creates object in memory	Auto called
__init__	Initializes object	def __init__(self)
self	Refers to current object	self.name
Instance variable	Unique to each object	self.name
Class variable	Shared by all objects	Dog.count

💬 **INTERVIEW ANSWER**

Q: What is instantiation in Python OOP?

A: Instantiation is the process of creating an object (instance) from a class.

When we instantiate a class, Python internally calls two methods:

1. __new__() → allocates memory and creates the object
2. __init__() → initializes the object with values

For example:

dog = Dog("Tommy", 3)

Here 'dog' is an instance of class Dog. Each instance gets its own copy of instance variables but shares class variables with other instances.

We can create multiple independent objects from the same class, each with their own state and data.

💡 **One line to remember: "A Class is a Cookie Cutter 🍪, Instantiation is pressing it into dough, and the Object is the Cookie you get!"**